

Correction — TD/TP 6

Classes abstraites et polymorphisme dynamique

MAM4 — Programmation C++

Cette correction donne le code final et les réponses essentielles. Le test temporaire avec `new/delete` ne doit pas rester dans le programme final.

1. Type statique et type dynamique

```
const Triangle triangle{7, 2.5};
const Element& element{triangle};
```

Réponse. Le type statique de `element` est `Element`. Son type dynamique est `Triangle`. La référence ne crée aucun nouvel objet. Avant `virtual`, l'appel `element.area()` utilise la fonction ordinaire de `Element` et renvoie 0.

2–3. Interface abstraite

`element.hpp` :

```
#pragma once

class Element {
public:
    int id() const;
    virtual double area() const = 0;
    virtual ~Element() = default;

protected:
    explicit Element(int id);

private:
    int id_;
};
```

`element.cpp` :

```
#include "element.hpp"

Element::Element(int id)
    : id_{id}
{}

int Element::id() const
{
    return id_;
}
```

Réponse. `= 0` rend `area()` virtuelle pure et `Element` abstraite. Son constructeur est `protected` : les classes dérivées peuvent construire leur sous-objet `Element`, mais le code extérieur ne peut pas créer directement un `Element`.

Réponse. `explicit` interdit l'utilisation implicite du constructeur pour convertir un entier en objet. L'appel direct `Element{id}` dans la liste d'initialisation reste autorisé.

4. Classes concrètes et override

```
class Triangle : public Element {
public:
    Triangle(int id, double area);
    double area() const override;

private:
    double area_;
};

class Quadrilateral : public Element {
public:
    Quadrilateral(int id, double area);
    double area() const override;

private:
    double area_;
};
```

```
Triangle::Triangle(int id, double area)
    : Element{id}, area_{area}
{}

double Triangle::area() const
{
    return area_;
}

Quadrilateral::Quadrilateral(int id, double area)
    : Element{id}, area_{area}
{}

double Quadrilateral::area() const
{
    return area_;
}
```

Réponse. Sans le `const`, `Triangle::area()` possède une autre signature et ne redéfinit pas la fonction virtuelle pure. Avec `override`, le compilateur signale l'erreur sur cette déclaration.

À retenir. Écrire `override` sur chaque redéfinition attendue fait vérifier la signature par le compilateur.

5. Destructeur virtuel

La déclaration correcte est déjà présente dans `Element` :

```
virtual ~Element() = default;
```

Réponse. Une destruction effectuée à travers `Element*` doit suivre le type réel de l'objet et appeler d'abord le destructeur dérivé. Sans destructeur virtuel dans la classe de base, le comportement est indéfini.

6. Un seul code client

report.cpp :

```
#include "report.hpp"

#include <iostream>

void report(const Element& element)
{
    std::cout << "element " << element.id()
               << " : aire = " << element.area() << '\n';
}

double total_area(const std::vector<const Element*>& elements)
{
    double total{};
    for (const Element* element : elements) {
        total += element->area();
    }
    return total;
}
```

Test final :

```
const Triangle triangle{7, 2.5};
const Quadrilateral quadrilateral{8, 4.0};

const std::vector<const Element*> elements{
    &triangle,
    &quadrilateral
};

assert(elements[0]->area() == 2.5);
assert(elements[1]->area() == 4.0);
assert(total_area(elements) == 6.5);
```

Réponse. Le vecteur contient deux pointeurs du même type `const Element*`. Ils désignent pourtant un **Triangle** et un **Quadrilateral**. Ces pointeurs ne possèdent pas les objets locaux.

7. Référence et slicing

Réponse. Une référence de classe de base désigne encore l'objet dérivé complet. Une copie par valeur vers une base concrète ne conserverait que la partie de base : c'est le slicing. Ici, **Element** est abstraite, donc cette copie est directement refusée.

À retenir. `virtual` choisit le comportement selon l'objet réel ; `override` vérifie la redéfinition ; le destructeur virtuel assure une destruction correcte à travers la classe de base.